Backtracking

Aula 07 - Análise e Técnicas de Algoritmos

Professor Daniel Marques

UEPB
Universidade
Estadual da Paraíba

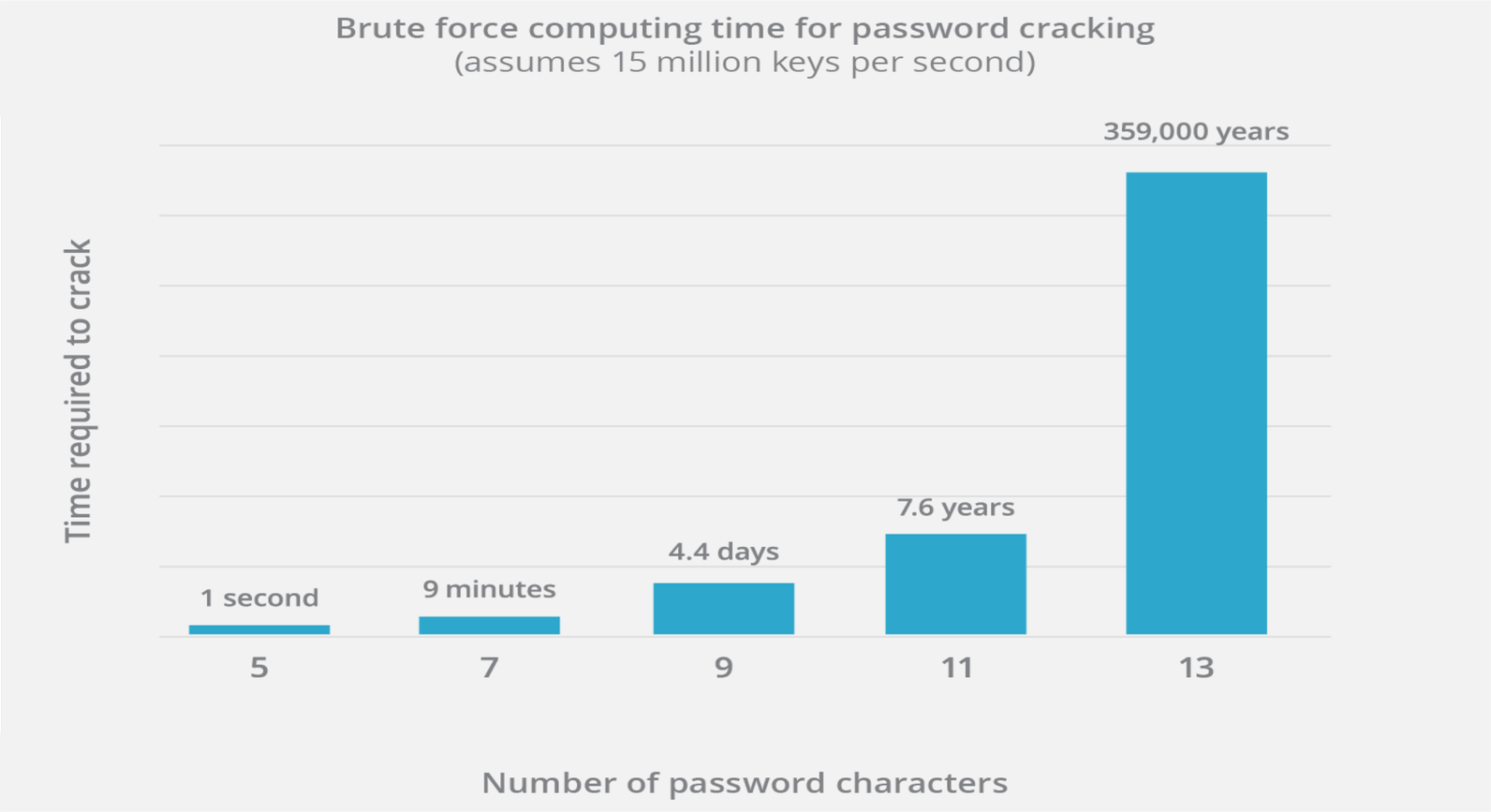
1

Introdução

Força bruta

Problemas de força bruta (busca combinatória) podem ser abordados por busca exaustiva

1. **Listar o espaço de busca**
2. **Examinar cada possibilidade**
3. **Retornar as soluções que satisfazem o problema**

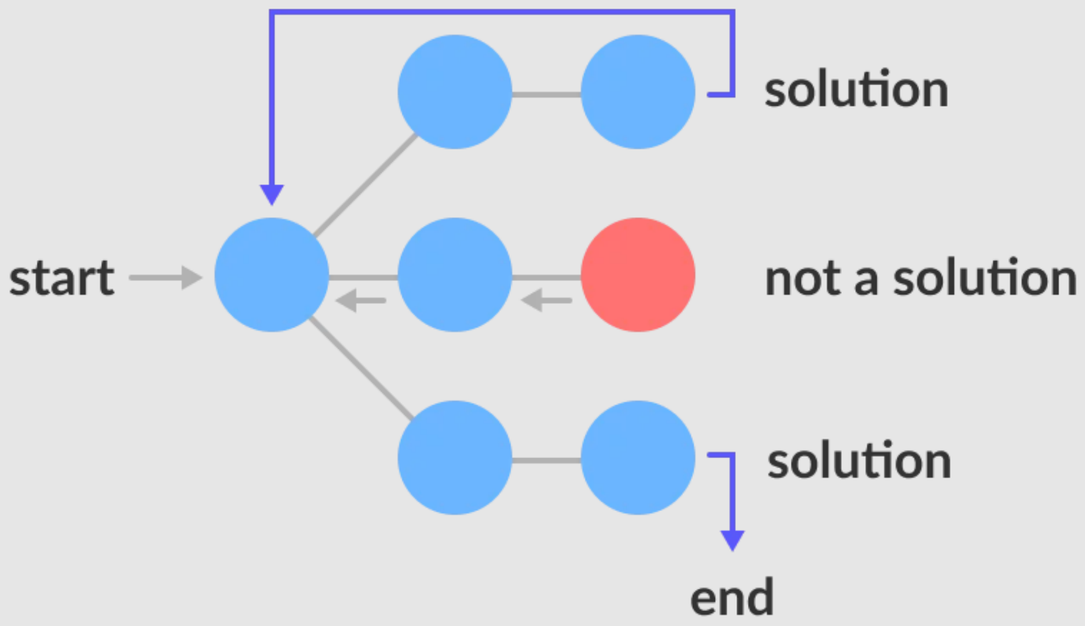


- **Ineficiente quando o espaço de busca é grande.**
- **Como fazer melhor?**

Backtracking

Aborda problemas complexos de força bruta

Utilizada para encontrar conjuntos que satisfaçam algumas restrições



Mais eficiente que força bruta: só gera soluções candidatas promissoras

Formulação do problema com backtracking

Os problemas abordados por Backtracking possuem a seguinte forma

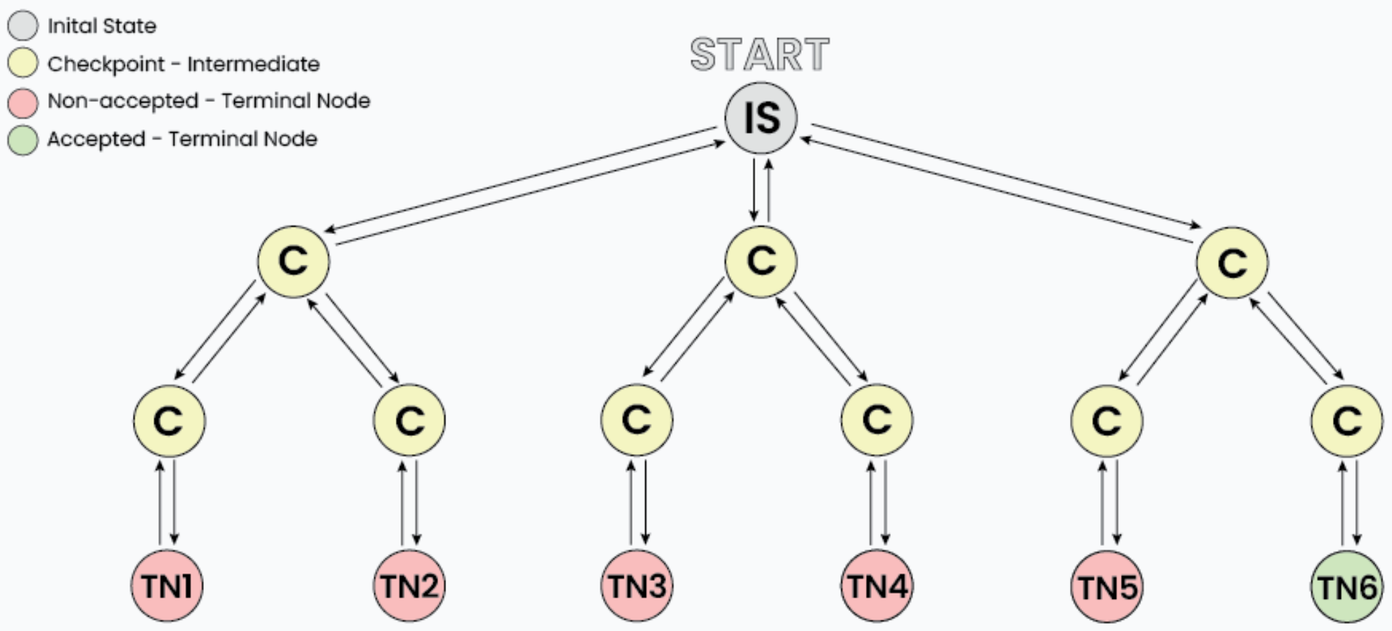
Problemas Abordados por Backtracking

Ache um subconjunto $S \subseteq A_1 \times A_2 \times \ldots \times A_n$ (A_i conjunto finito onde $1 \leq i \leq n$) tal que cada elemento $(s_1, s_2, \ldots, s_n) \in S$ satisfaça algumas restrições.

Ideia geral do backtracking

1. As soluções são construídas um componente por vez

2. Cada solução parcial é avaliada como promissora ou não promissora



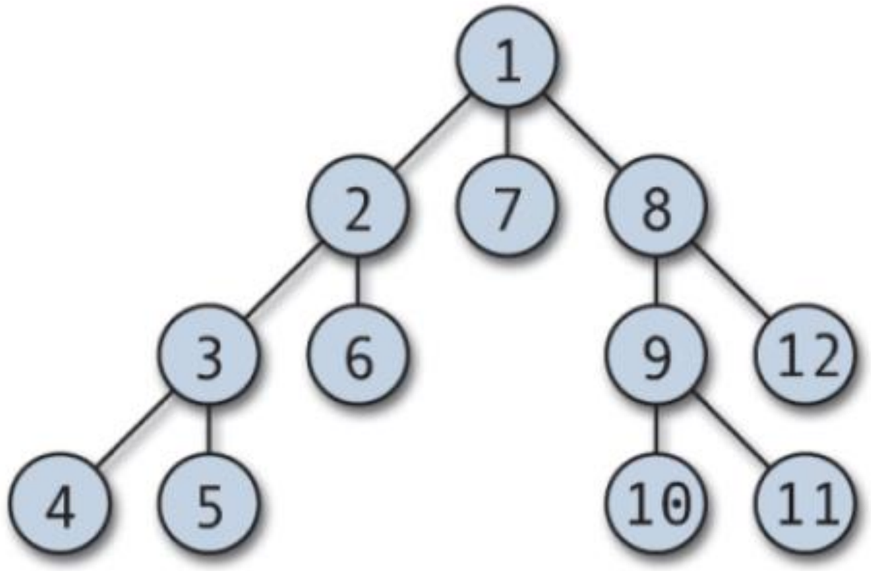
3. Se uma solução parcial é promissora, selecionamos o próximo componente da solução.

4. Senão, o algoritmo retrocede (backtrack) e substitui o último componente da solução parcial por sua próxima opção.

Árvore de estados

Backtracking induz uma árvore do espaço de soluções:

1. A raiz corresponde ao estado inicial (antes do início da busca por uma solução)



2. Um nó interno corresponde á uma solução parcial.

3. Um nó folha corresponde a uma solução parcial não promissora ou à solução final.

4. A busca é feita em profundidade (DFS).

2

Algoritmo de Backtracking

Algoritmo de backtracking

Estrutura geral do algoritmo

1. Defina a representação da solução (normalmente um array).
2. Estabeleça os conjuntos $A_1, \dots, A_n$ e a ordem em que seus elementos são processados.
3. Estabeleça as condições que tornam uma solução parcial válida
4. Estabeleça um critério para identificar a solução final

Estrutura geral do algoritmo

1. O algoritmo induz uma árvore do espaço de estados

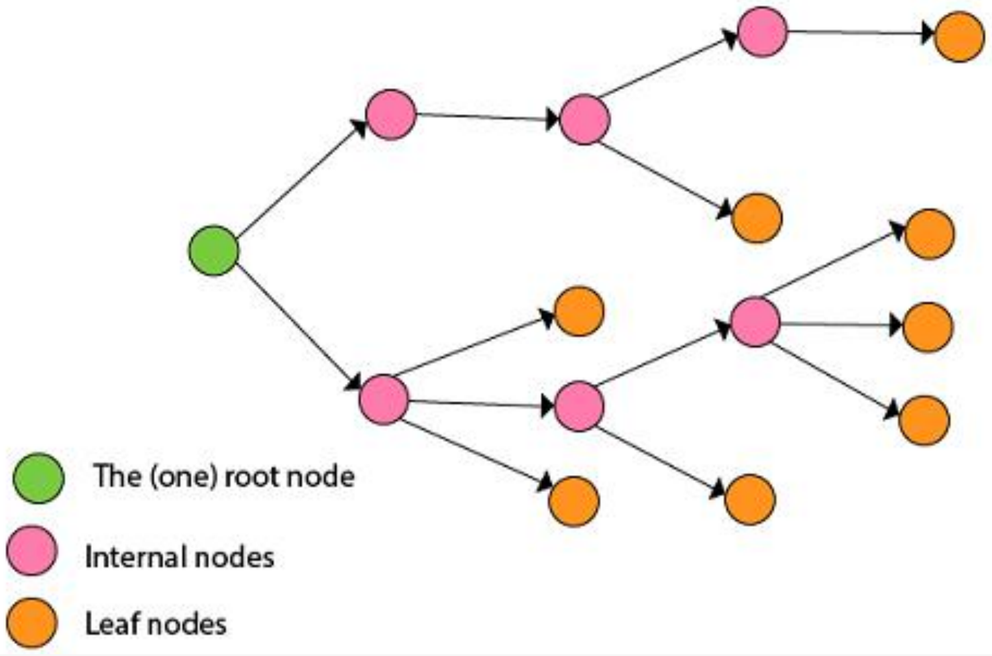
```
Backtrack(x)
    if x is not a solution
        return false
    if x is a new solution
        add to list of solutions
    backtrack(expand x)
```

2. Cada nó representa enuplas parciais com os primeiros i componentes da solução parcial

Estrutura geral do algoritmo

Para cada $(x_1, x_2, \dots, x_i)$ válida, o algoritmo acha o próximo elemento em A_{i+1} que é consistente com

- **Os valores de $(x_1, x_2, \dots, x_i)$**
- **As restrições do problema**



- **E adiciona à $(x_1, x_2, \dots, x_i)$ como seu $(i + 1)$ componente**

Algoritmo do Backtracking

```
BACKTRACK(X[1..i])
// Input: X[1..i] (i primeiros componentes da solução)
// Output: todas as enuplas representando as soluções do problema
if X[1..i] == solução
    write X[1..i]
else for each x ∈ A_{i+1} consistente com as restrições
        X[i+1] = x
        BACKTRACK(X[1..i+1])
```

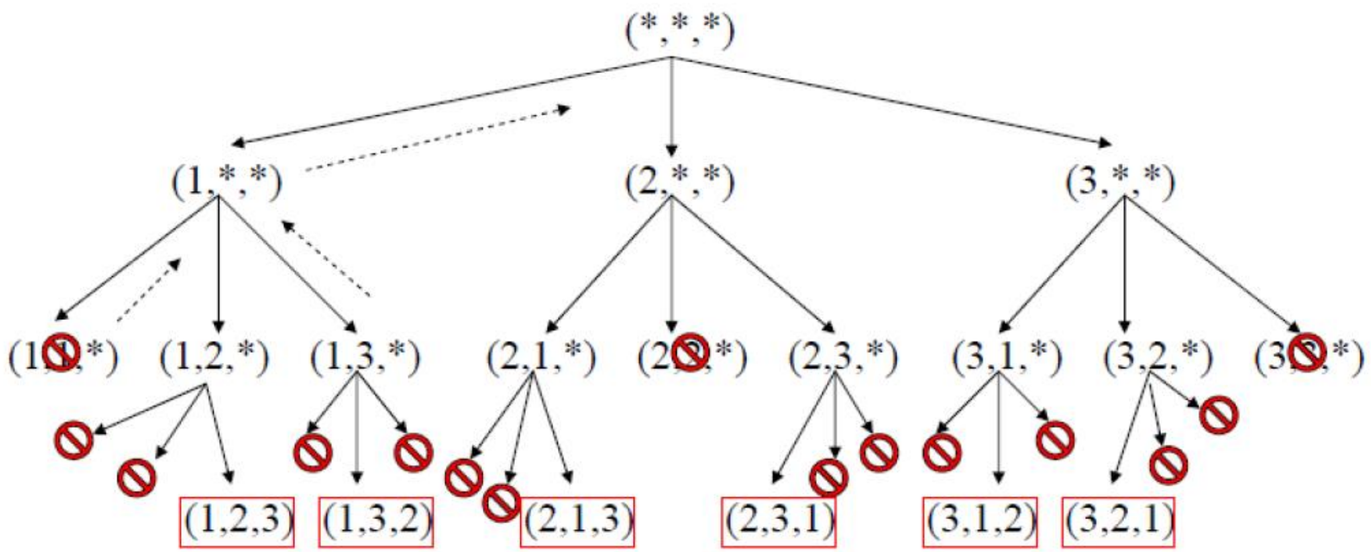
Para iniciar um algoritmo de backtracking, o seguinte pseudo-código pode ser chamado para $i = 0$ ($X[1..0]$ **representa a enupla vazia)**

3

Gerando permutações

Gerando permutações

Use Backtracking para gerar todas as permutações de $\{1, 2, 3\}$

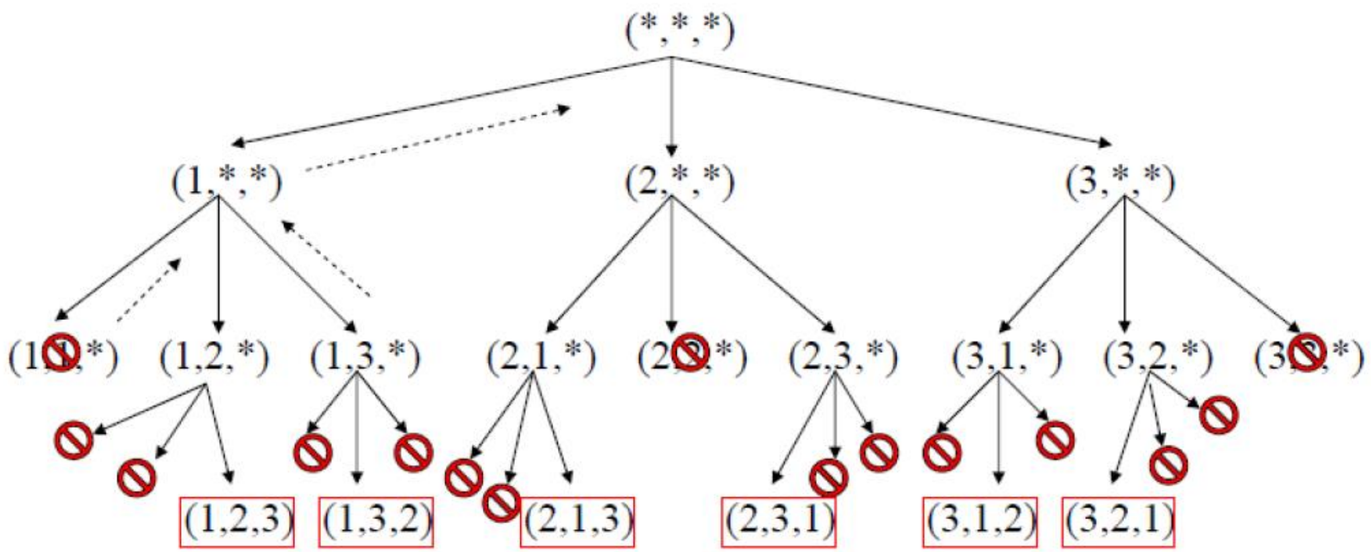


- $A_i = \{1, 2, 3\}$ **onde** $1 \leq i \leq 3$
- **Restrições:** $S_j \neq S_k$ **para todo** $S_j, S_k \in S$ **e todo** $j \neq k$

Gerando permutações

1. **Representação da solução: um array.**

$X[1..n]$ **satisfazendo** $X[i] \neq X[j]$ **para todo** $i \neq j$



2. **Condições de continuação: qualquer solução parcial** $X[1..i]$ **deve satisfazer** $X[i] \neq X[j]$ **para todo** $j < i$

3. **Critério de solução final:** $i == n$

```
PERM(X[1..i], A)
    // A == conjunto a ser permutado
    if i == n
        write X[1..n]
    else for each a ∈ A
            X[i+1] = a
            if VALID(X[1..i+1])
                PERM(X[1..i+1], A)
```

```
VALID(X[1..i])
    for j = 1 to i − 1
        if X[j] == X[i]
            return FALSE
    return TRUE
```

Gerando permutações

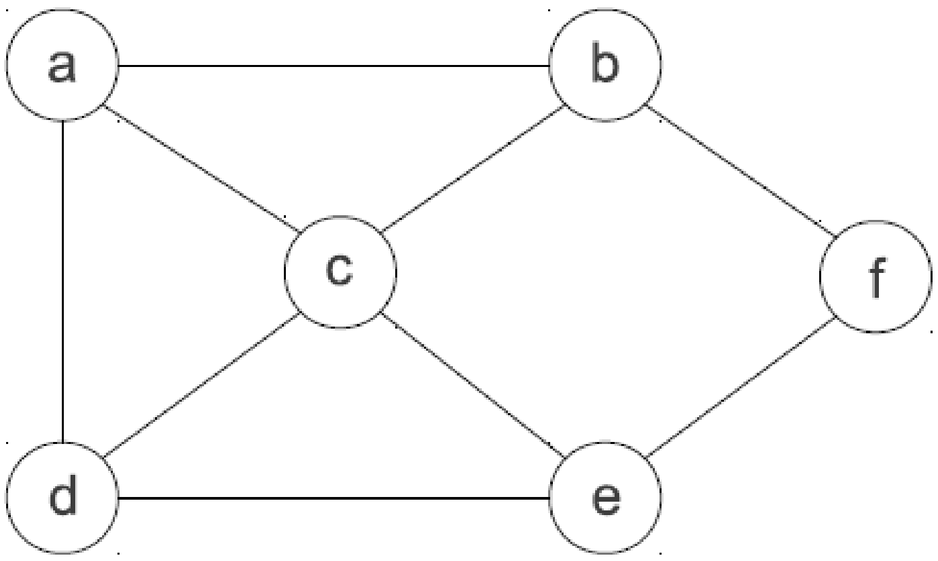
4

Problema do circuito hamiltoniano

Circuito Hamiltoniano

Definição do problema

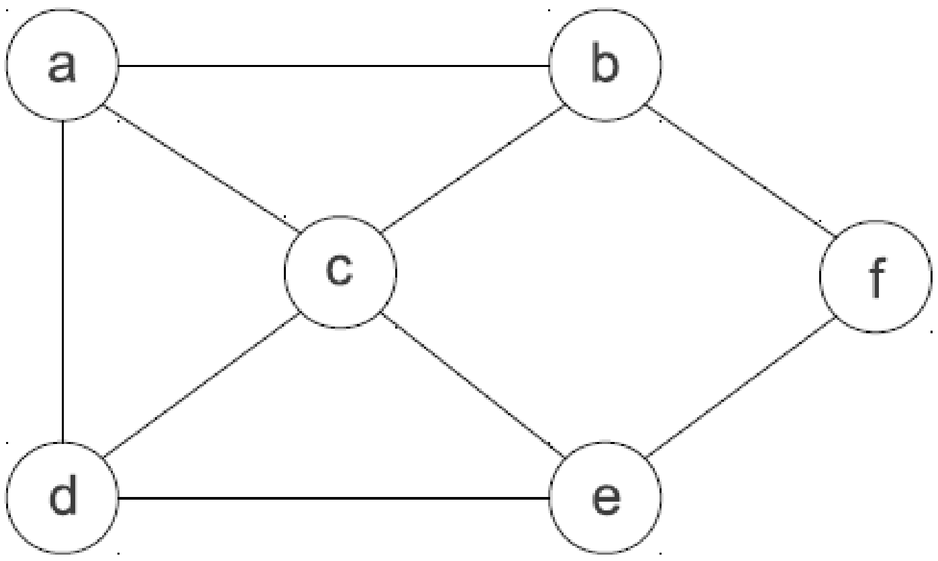
Um circuito hamiltoniano em um grafo não direcionado G = (V, E) é um ciclo simples que passa por todos vértices uma única vez.



- Entrada: grafo G com n vértices
- Saída: ciclo hamiltoniano

Circuito Hamiltoniano

$A_i = \{a, b, c, d, e, f\}$ **onde** $i \in \mathbb{N}$ **e** $1 \leq i \leq n + 1$



Restrições:

- $S_i \neq S_j$ **para todo** $S_i, S_j \in S$ **e todo** $i \neq j$ **com** $1 < i, j < n$
- $(S_i, S_{i-1}) \in S$ **e para todo** $1 \leq i \leq n + 1$ **(cada novo componente deve ter uma aresta com o componente anterior**

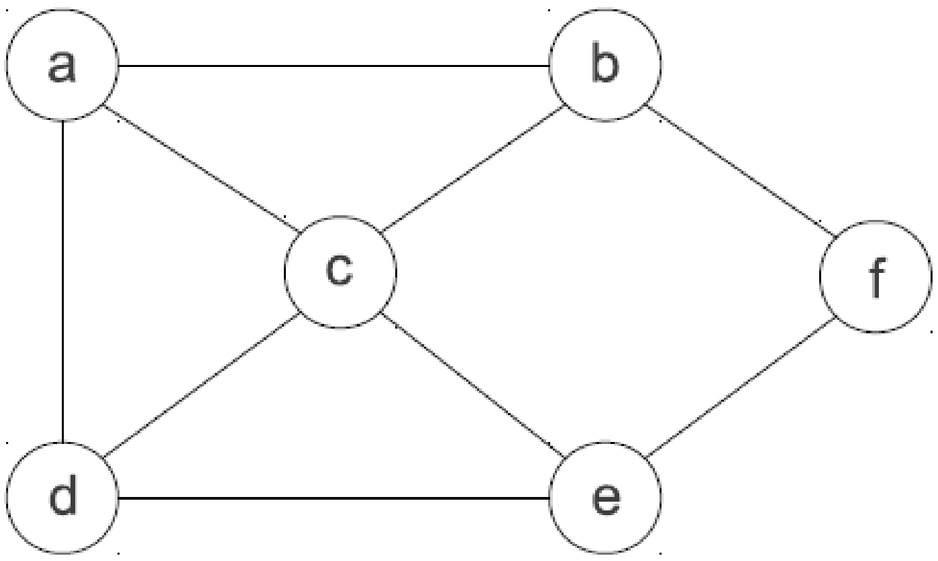
```
HAM(X[1..i], V, E, s)
    // Input: vertice V, aresta E e origem s ∈ V
    if i == n + 1
        write X[1..n]
    else for each v ∈ V
            X[i + 1] = v
            if VALID(X[1..i + 1], E)
                HAM(X[1..i + 1], V, E)
```

```
VALID(X[1..i])
    for j = 1 to i − 1
        if (X[j] == X[i]) and ((X[1] and X[n + 1]) ≠ s)
            return FALSE
    if (X[i], X[i − 1]) ∈ E
        return TRUE
    else return FALSE
```

Circuito Hamiltoniano

Circuito Hamiltoniano

Desenhe a árvore de busca do backtracking para o grafo abaixo

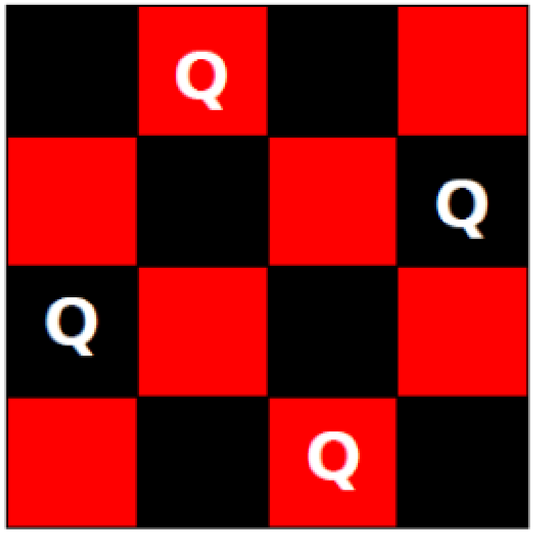


5

Problema das N-Rainhas

Problema das N-Rainhas

Ache todas as possibilidades de colocar n rainhas num tabuleiro de xadrez $n \times n$ tal que elas não possam se atacar:

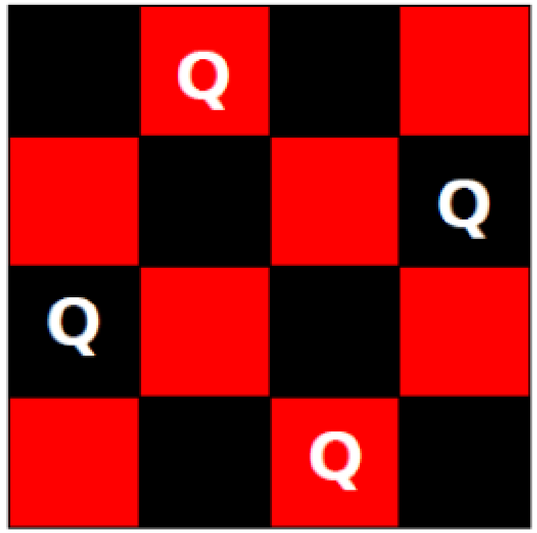


- cada linha contém apenas uma rainha
- cada coluna contém apenas uma rainha
- cada diagonal contém apenas uma rainha

UEPB
Universidade
Estadual da Paraíba

Problema das N-Rainhas

Quantos candidatos teríamos por força bruta (busca exaustiva) para n = 4?



- $16^4 = 65.536$ possíveis formas de dispor 4 rainhas no tabuleiro
- Uma busca exaustiva (força bruta) mais eficiente, coloca uma rainha em cada linha e portanto $4^4 = 256$ possibilidades de dispor 4 rainhas no tabuleiro

UEPB
Universidade
Estadual da Paraíba

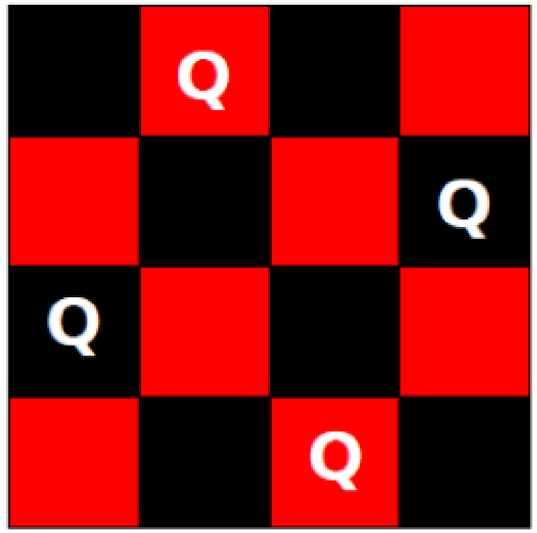
```
N-QUEENS(X[1..i])
if i == n
    write X[1..n]
else for j = 1 to n
        X[i+1] = j
        if VALID(X[1..i+1])
            N-QUEENS(X[1..i+1])
```

```
VALID(X[1..i])
for j = 1 to i-1
    if (X[j] == X[i]) or (|X[i] - X[j]| == |i - j|)
        return FALSE
    return TRUE
```

N-Rainhas

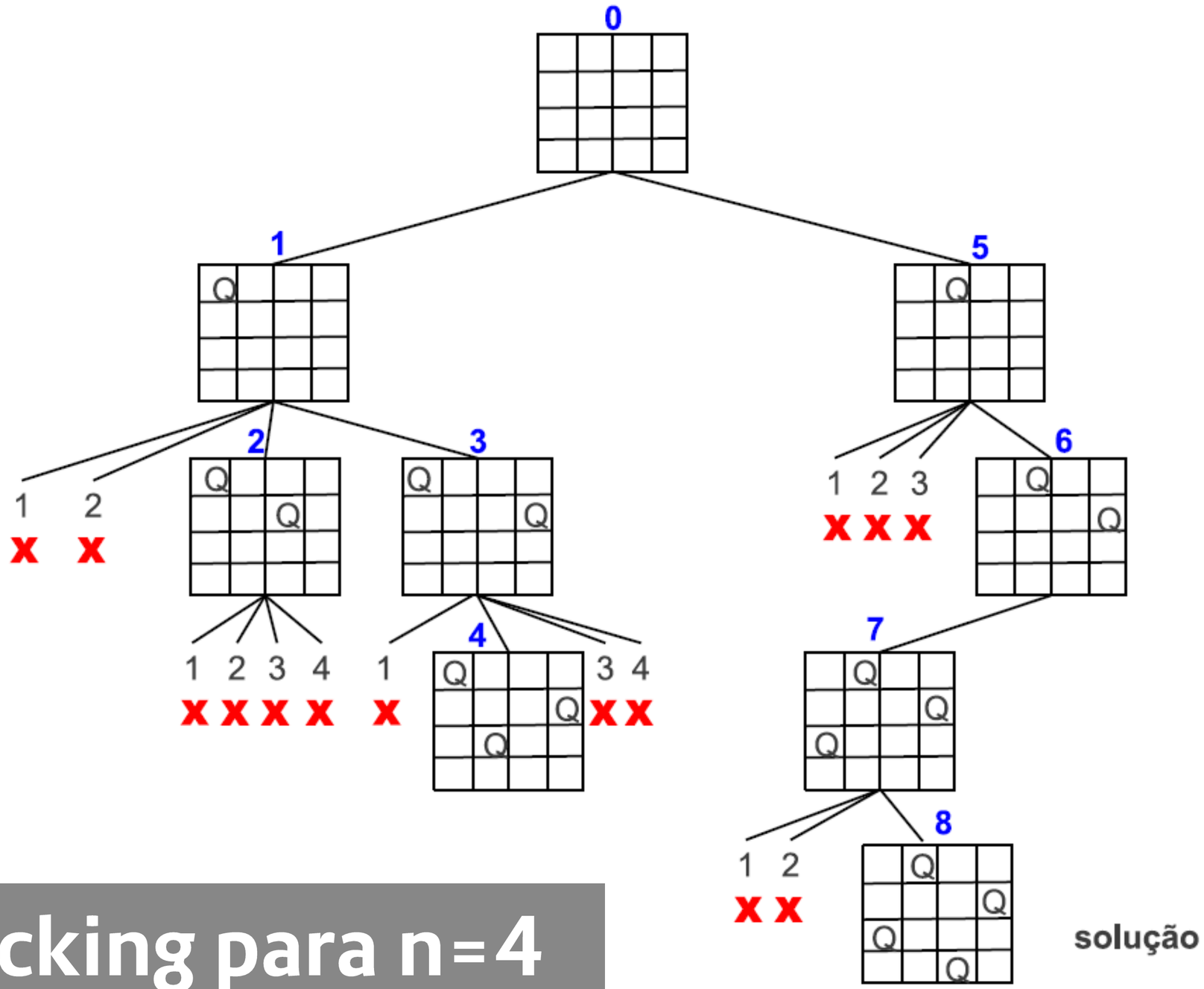
Problema das N-Rainhas

1. Representação da solução: um array $X[1..n]$ onde os índices representam linhas e valores e valores colunas



2. Condições de continuação: qualquer solução parcial $X[1..i]$ deve satisfazer:
    - 2.1. rainhas devem estar em colunas diferentes
    - 2.2. rainhas não podem estar na mesma diagonal
3. Critério solução final: $i == n$

UEPB
Universidade
Estadual da Paraíba

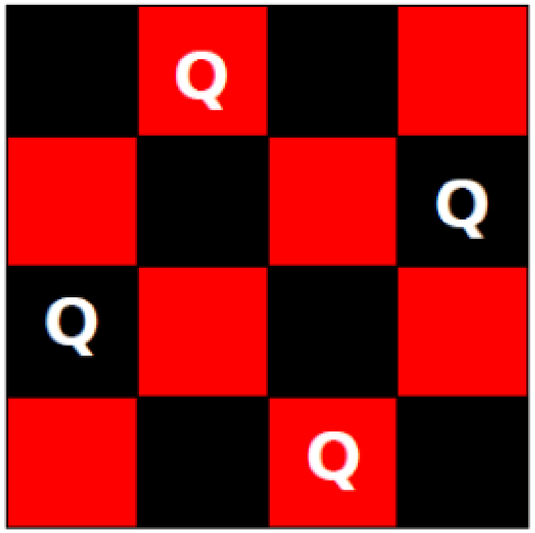


Backtracking para n=4

N-Rainhas

Exercício 2

Explique como a simetria do tabuleiro pode ser usada para achar a segunda solução do problema das 4-rainhas.



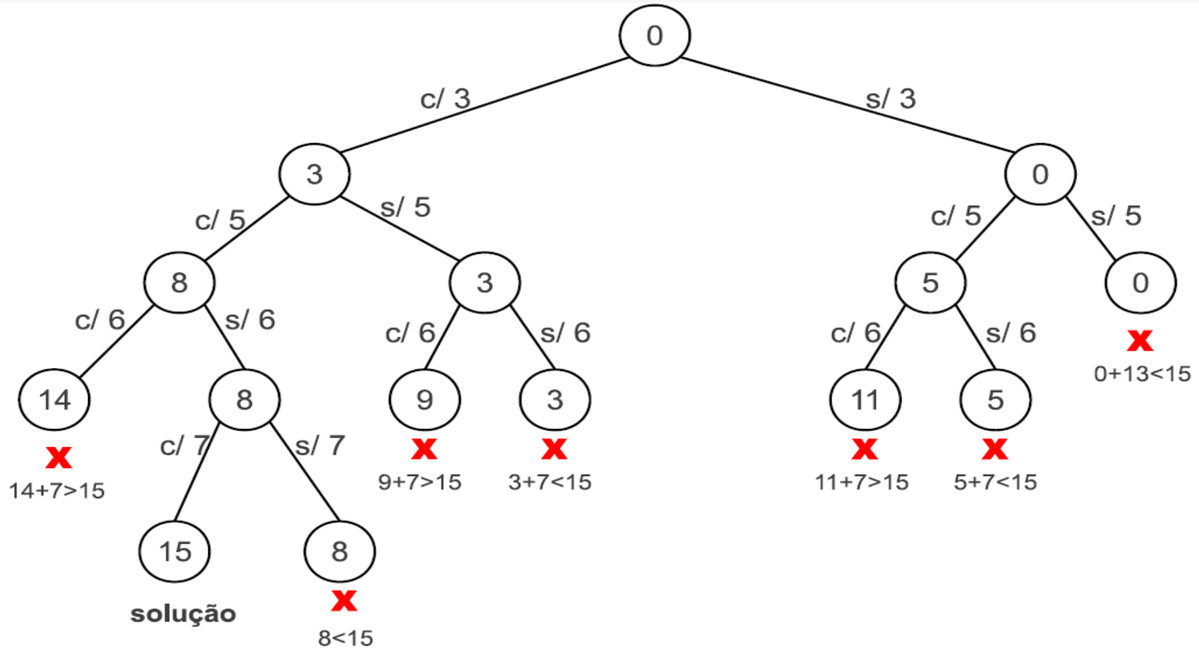
Ao lado encontra-se a primeira solução mostrada no slide anterior

6

Problema da soma do subconjunto

Problema da soma do subconjunto

Achar o subconjunto de um dado conjunto $B = \{b_1, \dots, b_n\}$ de n inteiros positivos cujo a soma de seus elementos é igual a um dado inteiro positivo d



- Por exemplo, para $B = \{1, 2, 5, 6, 8\}$ e $d = 9$, há duas soluções: $\{1, 2, 6\}$ e $\{1, 8\}$
- É conveniente ordenar os elementos do conjunto em ordem crescente: $b_1 \leq b_2 \leq \cdots \; b_n$

UEPB
Universidade
Estadual da Paraíba

Problema da soma do subconjunto

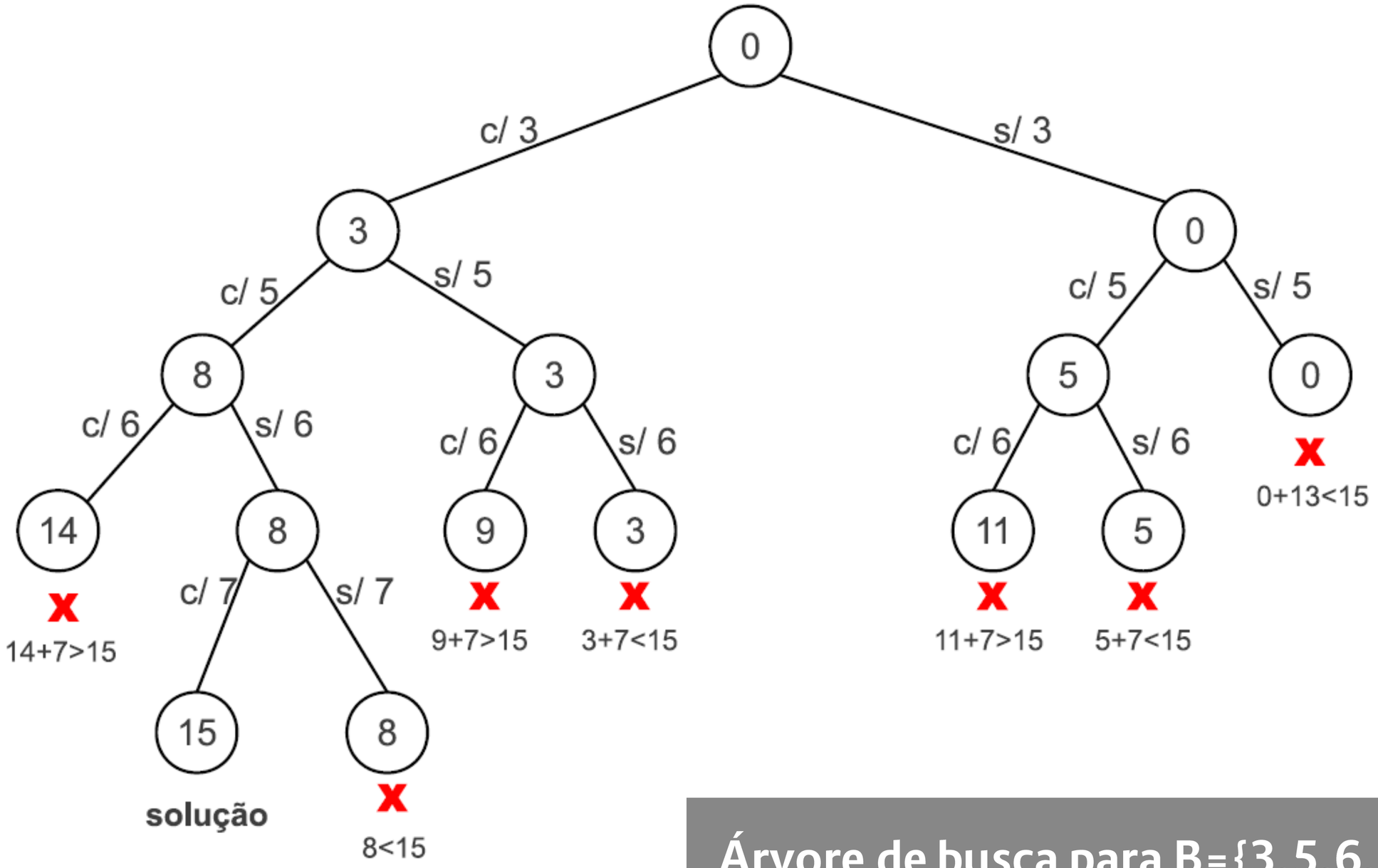
O espaço de estados é representado por uma árvore binária onde:

- a raiz marca o início (nenhum elemento escolhido)
- Nós a direita e esquerda representam inclusão e exclusão de elementos.

$$s' + s_{i+1} > d \quad \text{soma muito grande}$$

$$s' + \sum_{j=i+1}^{n} s_j < d \quad \text{soma muito pequena}$$

- Caminhos da raiz até o i-ésimo nível indicam os elementos escolhidos.
- As somas parciais b' são armazenadas nos nós.



Árvore de busca para B={3,5,6,7} e d=15

Considerações Finais

Busca combinatória	1	Adequado a problemas complexos de busca combinatória.
Similar a força bruta	2	No pior caso pode ter que gerar todos os candidatos (igual força bruta).
Poda	3	A melhora depende da capacidade de poda de soluções não promissoras
Análise assintótica	4	Muito difícil de analisar assintoticamente (depende do tamanho da árvore de estados).
The end	5	

UEPB
Universidade
Estadual da Paraíba

Dúvidas, sugestões ou reclamações?

• • •

Alguma pergunta?

UEPB
Universidade
Estadual da Paraíba

Referências

1. MARINHO, Leandro Balby. **Backtracking**. Universidade Federal de Campina Grande. 2018.

2. LEVITIN, Anany. **Introduction to the Design and Analysis of Algorithms**. Segunda Edição. Pearson International Edition, 2007.

3.

4.

5.